

FieldFlow

SDK Guide

FieldFlow SDK Guide

This guide shows how to integrate with the FieldFlow API from your own application. Every example is runnable — copy, set your environment variables, and go.

Companion document: [API.md](#) lists every endpoint. **SDK.md** (this file) shows you *how to use them*.

1. Overview

The FieldFlow API is REST + JSON over HTTPS, with Server-Sent Events (SSE) for AI streaming endpoints. There is no published SDK package — you call the API directly with a standard HTTP client. This guide gives you idiomatic patterns for the two languages we recommend:

- **JavaScript/TypeScript** (browser or Node) — using `fetch` plus the Firebase Auth client SDK
- **Python** — using `requests` plus `firebase-admin`

All endpoints live under `https://chatbot-492915.el.r.appspot.com/api/`.

2. Quick start

2.1 Install dependencies

JavaScript / TypeScript:

```
npm install firebase
```

Python:

```
pip install firebase-admin requests
```

2.2 Get a Bearer token

FieldFlow accepts a Firebase ID token in the `Authorization: Bearer` header. The token comes from the Firebase Auth client SDK after a successful login.

JavaScript (browser, using Firebase Web SDK):

```
import { initializeApp } from 'firebase/app'
import { getAuth, signInWithEmailAndPassword } from 'firebase/auth'

const app = initializeApp({
```

```
apiKey: '<VITE_FIREBASE_API_KEY>',
authDomain: '<project>.firebaseapp.com',
projectId: '<project>',
})

const auth = getAuth(app)
await signInWithEmailAndPassword(auth, email, password)
const token = await auth.currentUser.getIdToken()
```

Python (server-side, using a service account):

```
import firebase_admin
from firebase_admin import credentials, auth

cred = credentials.Certificate('path/to/service-account.json')
firebase_admin.initialize_app(cred)

# Create a custom token for a user, exchange for an ID token via REST
uid = 'user_abc'
custom_token = auth.create_custom_token(uid).decode('utf-8')
# Then exchange via Identity Toolkit:
import requests
r = requests.post(
    f'https://identitytoolkit.googleapis.com/v1/accounts:signInWithCustomToken?key=<API_KEY>',
    json={'token': custom_token, 'returnSecureToken': True},
)
id_token = r.json()['idToken']
```

2.3 Call any endpoint

JavaScript:

```
const res = await fetch('https://chatbot-492915.el.r.appspot.com/api/action-plans', {
  headers: { Authorization: `Bearer ${token}` },
})
const { plans } = await res.json()
```

Python:

```
res = requests.get(
    'https://chatbot-492915.el.r.appspot.com/api/action-plans',
    headers={'Authorization': f'Bearer {id_token}'},
)
plans = res.json()['plans']
```

curl:

```
curl https://chatbot-492915.el.r.appspot.com/api/action-plans \
-H "Authorization: Bearer $TOKEN"
```

3. The recommended client wrapper

In your own code, wrap the auth handling once so the rest of your app doesn't repeat it.

3.1 JavaScript helper

```
// fieldflow.ts
const BASE = 'https://chatbot-492915.el.r.appspot.com/api'

async function getToken(): Promise<string> {
  const { getAuth } = await import('firebase/auth')
  const auth = getAuth()
  if (!auth.currentUser) throw new Error('Not logged in')
  return auth.currentUser.getIdToken()
}

export async function call(path: string, opts: RequestInit = {}) {
  const token = await getToken()
  const res = await fetch(`${BASE}${path}`, {
    ...opts,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
      ...opts.headers,
    },
  })
  if (!res.ok) {
    const err = await res.json().catch(() => ({}))
    throw new Error(err.error || `HTTP ${res.status}`)
  }
  return res.json()
}

// Convenience methods
export const ff = {
  get: (p: string) => call(p),
  post: (p: string, body: any) => call(p, { method: 'POST', body: JSON.stringify(body) }),
  put: (p: string, body: any) => call(p, { method: 'PUT', body: JSON.stringify(body) }),
  patch: (p: string, body: any) => call(p, { method: 'PATCH', body: JSON.stringify(body) }),
  delete: (p: string) => call(p, { method: 'DELETE' }),
}
```

3.2 Python helper

```
# fieldflow.py
import requests
from firebase_admin import auth as fb_auth

BASE = 'https://chatbot-492915.el.r.appspot.com/api'

class FieldFlow:
    def __init__(self, id_token: str):
```

```
self.token = id_token

def _headers(self):
    return {
        'Authorization': f'Bearer {self.token}',
        'Content-Type': 'application/json',
    }

def get(self, path, **params):
    r = requests.get(f'{BASE}{path}', headers=self._headers(), params=params)
    r.raise_for_status()
    return r.json()

def post(self, path, data):
    r = requests.post(f'{BASE}{path}', headers=self._headers(), json=data)
    r.raise_for_status()
    return r.json()

def put(self, path, data):
    r = requests.put(f'{BASE}{path}', headers=self._headers(), json=data)
    r.raise_for_status()
    return r.json()

def delete(self, path):
    r = requests.delete(f'{BASE}{path}', headers=self._headers())
    r.raise_for_status()
    return r.json()
```

4. Common patterns

4.1 Paginated lists

Most list endpoints accept `?limit=` and `?offset=` plus an optional `?q=` search:

```
const { contacts } = await ff.get('/wa/contacts?limit=50&offset=0&q=ramesh')
```

4.2 Soft-deleted records

Some resources (action plans) soft-delete by default and need `?include_archived=1` to show:

```
const { plans } = await ff.get('/action-plans?include_archived=1')
```

4.3 Server-Sent Events (streaming AI responses)

The `fetch` API in browsers returns a `ReadableStream`. Parse line-by-line:

```
async function streamChat(message: string, sources: any[]) {
    const token = await getToken()
```

```

const res = await fetch(`${BASE}/notebook/chat`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${token}`,
  },
  body: JSON.stringify({ message, sources, history: [] }),
})
const reader = res.body!.getReader()
const decoder = new TextDecoder()
let buf = ''
while (true) {
  const { value, done } = await reader.read()
  if (done) break
  buf += decoder.decode(value, { stream: true })
  const lines = buf.split('\n')
  buf = lines.pop() ?? ''
  for (const line of lines) {
    if (!line.startsWith('data: ')) continue
    const raw = line.slice(6).trim()
    if (raw === '[DONE]') return
    const chunk = JSON.parse(raw)
    if (chunk.text) process.stdout.write(chunk.text) // or update UI
  }
}
}

```

Python (using `sseclient-py`):

```

import requests
from sseclient import SSEClient

r = requests.post(f'{BASE}/notebook/chat', headers=headers,
                  json={'message': 'Hi', 'sources': []},
                  stream=True)
for evt in SSEClient(r).events():
    if evt.data == '[DONE]': break
    chunk = json.loads(evt.data)
    if 'text' in chunk:
        print(chunk['text'], end='', flush=True)

```

4.4 Error handling

Every error response is JSON:

```
{ "error": "Plan not found", "cid": "abc12345" }
```

Use the `cid` when reporting issues — it's the correlation ID server logs are indexed by.

In production, `e.message` is **not** returned (only `cid` + generic "Internal server error"). In development, both are returned for easier debugging.

4.5 Rate-limit handling

When you exceed a rate limit you get `429 Too Many Requests with Retry-After: <seconds>`:

```
async function callWithBackoff(path: string, opts: RequestInit = {}, tries = 3) {
  for (let i = 0; i < tries; i++) {
    const res = await fetch(`${BASE}${path}`, { ...opts, headers: { ...opts.headers } })
  }
  if (res.status !== 429) return res
  const wait = parseInt(res.headers.get('retry-after') || '5') * 1000
  await new Promise(r => setTimeout(r, wait))
}
throw new Error('Rate limit exceeded after retries')
}
```

5. Integration recipes

5.1 Submit a daily report from a partner mobile app

```
const text = 'Today visited Khagaria, met 30 farmers, training on pen culture'

// Step 1 – AI extracts structured fields
const { fields, confidence } = await ff.post('/reports/quick-extract', {
  text,
  projectsHint: ['Kosi Sahjivan'],
})
// fields = { location: 'Khagaria', state: null, project: 'Kosi Sahjivan',
//           area_of_intervention: 'training', beneficiaries: 30, description: '...'
// }

// Step 2 – Show fields to user, let them edit, then submit
const result = await ff.post('/reports/quick-submit', {
  fields,
  original_text: text,
  photo_urls: [],
})
// result.id = newly inserted daily_reports row
```

5.2 Upload an Action Plan from Excel

```
import * as XLSX from 'xlsx'

const file = (await fileInput.files[0]).arrayBuffer()
const wb = XLSX.read(await file, { type: 'array' })

// Parse Plan Info sheet
const info = XLSX.utils.sheet_to_json(wb.Sheets['Plan Info'], { header: 1 })
// Parse Monthly Plan sheet (S.N. / Activity / Category / Apr-T / Location / May-T/A / ...)
// See docs/SDK.md §5.2 in your own client for the full Kosi-format parser.

await ff.post('/action-plans', {
  name: 'JJM Bihar 2026',
```

```

    year:      2026,
    start_month: 4,
    locations:  ['Patna', 'Gaya', 'Muzaffarpur'],
    activities: parsedActivities,
  })

```

5.3 Send a WhatsApp message to one contact

```

// Look up contact
const { contacts } = await ff.get('/wa/contacts?q=ramesh')
const contact = contacts[0]

// Send
const { message } = await ff.post(`/wa/contacts/${contact.id}/send`, {
  text: 'Your training is confirmed for next Tuesday at 10am.',
})
console.log('Sent message ID:', message.id)

```

5.4 Receive WhatsApp messages via webhook

If you want a copy of every inbound WA message to flow into your own system, register your URL with Meta Business Manager (NOT with FieldFlow — FieldFlow uses its own webhook URL). Then verify signatures server-side:

```

// Express example
import crypto from 'crypto'
import express from 'express'

const app = express()
const APP_SECRET = process.env.YOUR_META_APP_SECRET

app.post('/your-webhook', express.json({
  verify: (req, _res, buf) => { req.rawBody = buf },
}), (req, res) => {
  const sig = req.header('X-Hub-Signature-256') || ''
  const expected = 'sha256=' + crypto.createHmac('sha256', APP_SECRET)
    .update(req.rawBody).digest('hex')
  if (!crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected))) {
    return res.status(401).end()
  }
  // Process req.body...
  res.sendStatus(200)
})

```

5.5 Generate an AI report and stream tokens

```

const res = await fetch(`${BASE}/get-ai-report`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${token}` },
  body: JSON.stringify({
    reportType: 'project',
    filters: { project: ['Kosi Sahjivan'], dateFrom: '2026-04-01', dateTo: '2026-06-30' },
  })

```

```

    instruction: 'Quarterly Q1 report for donor (formal tone, 600 words)',
    language:    'en',
  }},
})

// Parse SSE (see §4.3 above)

```

5.6 Programmatically create a WhatsApp flow

```

const flow = await ff.post('/wa/flows', {
  name: 'Welcome Survey',
  description: 'Onboards new farmers – collects name, village, crop',
  trigger_keywords: ['start', 'hi'],
  show_progress: true,
  nodes: [
    { id: 'q1', type: 'wait_input', text: 'What is your name?',      save_as: 'name',
      next: 'q2' },
    { id: 'q2', type: 'wait_input', text: 'Which village?',          save_as: 'village',
      next: 'q3' },
    { id: 'q3', type: 'send_buttons', text: 'Main crop?', buttons: [
      { id: 'pen',      title: 'Pen culture' },
      { id: 'makhana', title: 'Makhana' },
      { id: 'other',    title: 'Other' },
    ], save_as: 'crop', next: 'recap' },
    { id: 'recap', type: 'recap_confirm', intro_text: "Here's what I captured:",
      yes_label: 'YES', edit_label: 'EDIT', next: 'end', next_edit: 'q1' },
    { id: 'end', type: 'end', text: 'Thanks – saved!' },
  ],
})
console.log('Flow created:', flow.id)

// Publish so it starts handling inbound messages
await ff.post(`/wa/flows/${flow.id}/publish`, { active: true })

```

5.7 Use AI to generate a flow

For internal tooling, the platform exposes a Gemini-backed flow generator:

```

const draft = await ff.post('/wa/flows/generate', {
  prompt: 'Build a WASH baseline survey for fish farmers. Ask name, village, pond size
, and main crop. Then thank them and end.',
})
// draft = { name, description, trigger_keywords, nodes[] }

// Save the draft
await ff.post('/wa/flows', draft)

```

5.8 Broadcast to a collection

```

// 1. See available collections
const { collections } = await ff.get('/wa/collections')
// e.g. [{ name: 'farmers-bihar', member_count: 247, opted_in_count: 220 }]

```

```
// 2. Send
const result = await ff.post('/wa/broadcast', {
  name: 'May Health Camp Reminder',
  message_body: 'Reminder: Health camp this Friday 10am at the village center.',
  recipient_filter: { tags: ['farmers-bihar'] },
})
// { broadcastId, total, status: 'sending' }

// 3. Track delivery via analytics
const { broadcasts } = await ff.get('/wa/analytics')
// broadcasts[0] has total_sent + total_failed
```

5.9 Aggregate Impact Dashboard data

```
const impact = await ff.get('/action-plans/impact')
// {
//   totals: { projects_count, target, achieved, pct, beneficiaries_reached },
//   projects: [...],
//   by_category: [...],
//   by_location: [...],
//   by_month: [...],
//   activities: [...],
// }
```

6. Role-based access

Your token's `role` claim controls which endpoints you can call:

Role	Can call
superadmin	Everything, including <code>/api/superadmin/*</code>
admin	Org configuration, all manager-level routes
manager	Reports, Action Plan (read/write), Impact, WhatsApp (read), Worker analytics
employee	Field reports submission, own profile

A 403 response means your role doesn't have access to that endpoint.

7. Webhook receivers (incoming events from FieldFlow)

FieldFlow does not yet send outbound webhooks. If you need event notifications (e.g. "WA flow completed → call my system"), the `webhook` flow node can POST to your URL during a conversation:

```
# In your flow definition
{
  id: 'webhook-1',
```

```
type: 'webhook',
method: 'POST',
url: 'https://your-system.example.com/fieldflow-events',
body: {
  farmer_name: '{{name}}',
  village: '{{village}}',
  crop: '{{crop}}',
},
save_as: 'response', // saves the response body
next: 'next-node',
next_on_error: 'fallback-node',
}
```

Your endpoint receives:

```
{
  "farmer_name": "Ramesh Kumar",
  "village": "Khagaria",
  "crop": "Pen culture",
  "_contact": {
    "wa_id": "919876543210",
    "name": "Ramesh"
  }
}
```

Respond with JSON `{ status: 'ok' }` or any object — its keys get flattened into the flow's collected variables.

8. Common gotchas

- **Tokens expire after 1 hour.** Get a fresh one with `auth.currentUser.getIdToken( / forceRefresh / true)` and retry the request.
- **SSE responses are not JSON.** Don't call `.json()` on a streaming response — read with `body.getReader()`.
- **Bulk endpoints have caps.** `/api/action-plan/progress/bulk` rejects > 5000 indicators per request — chunk client-side if you need more.
- **WhatsApp send endpoints require `wa_config` to be set.** A 400 "WhatsApp not configured" means your org hasn't run the one-time setup in the dashboard's WhatsApp → Settings tab.
- **NUMERIC values come back as strings.** Postgres NUMERIC columns (`planned`, `achieved`) serialise as strings in JSON. Coerce with `Number(...)` before arithmetic — or just rely on the `toNum` pattern we use internally.
- **Plan delete is soft by default.** Use `?purge=1` for hard delete. The `/api/action-plans/:id/restore` endpoint brings back a soft-deleted plan within the 30-day grace period.

9. End-to-end example

A complete Python script that submits a WhatsApp-driven field report:

```
from fieldflow import FieldFlow
import time

ff = FieldFlow(id_token=os.environ['FF_TOKEN'])

# Step 1 – Get the active action plan
plans = ff.get('/action-plans')['plans']
plan = plans[0]
print(f'Working on plan: {plan["name"]}')

# Step 2 – Submit a field report via Quick Report API
text = '''Today visited Bhaptiyahi village, ran a half-day training
session for 45 fish farmers on pen culture techniques. Two
farmers signed up for the cooperative.'''

extracted = ff.post('/reports/quick-extract', {
    'text': text,
    'projectsHint': [plan['name']],
})
print(f'AI extracted with {extracted["confidence"]*100:.0f}% confidence:')
print(f"  Location: {extracted['fields']['location']}")
print(f"  Beneficiaries: {extracted['fields']['beneficiaries']}")

# Step 3 – Submit it
result = ff.post('/reports/quick-submit', {
    'fields': extracted['fields'],
    'original_text': text,
})
print(f'Report saved: id={result["id"]}')

# Step 4 – Update the corresponding Action Plan cell
ff.put('/action-plan/progress?plan=' + plan['project_key'], {
    'indicator': '3|Bhaptiyahi|May', # activity 3, location Bhaptiyahi, May
    'achieved': 45,
    'notes': 'Training delivered as planned. 2 farmers joined cooperative.',
})
print('Action plan cell updated.')

# Step 5 – Pull the latest impact summary
impact = ff.get('/action-plans/impact')
print(f"Annual progress: {impact['totals']['pct']}%")
```

10. SDKs and libraries

There is **no published npm/PyPI SDK package** yet. Use the helpers above. If you want one, the patterns in §3 are stable; you can copy them into your own internal package.

If FieldFlow ships a published SDK in the future, the package names will be:

- `@fieldflow/sdk` (npm, TypeScript)
- `fieldflow` (PyPI)

Both will preserve the API surface documented here.

11. Versioning

The API is currently unversioned (no `/v1/` prefix). Breaking changes follow a deprecation cycle:

1. New behavior ships behind a flag or new endpoint.
2. Old endpoint returns `Warning: 299 - "deprecated, use X"` header for at least one release.
3. Old endpoint removed only after consumers have had time to migrate.

If you write code against this API, add a CI check that parses the `Warning` header on every response and surfaces deprecations to your team.

12. Getting help

- **Logs:** every error response includes a `cid`. Pass that to support and they can find the exact request in Cloud Logging.
- **Status page:** `GET /api/whatsapp/health` returns `{ ok: true }` if the platform is up.
- **API doc:** see [API.md](#) for the full endpoint catalogue.